

Game Jam #3 - Make sure to delegate 구현 디테일

1. 설계 목표 및 구조 개요

이번 게임잼의 핵심 목표는 지금까지 제작한 **Component**를 실제 게임 규칙에 연결하고, 게임 로직을 가능한 한 **Lua 스크립트로 위임**하는 것이었습니다. 저희 팀이 제작한 게임은 낯고 더러운 방을 정리하는 1인칭 청소 및 분류 게임입니다. 플레이어는 방 안의 오염을 제거하고, 발견한 물건을 보관하거나 버리는 결정을 내립니다.

C++ 엔진 코어는 Actor, Component, 충돌, UI, 오디오, 데이터 로딩 등 기반 기능을 제공하며, 각 사물의 반응과 구체적인 게임 규칙은 Lua와 Game 폴더 내 시스템이 담당하도록 역할을 명확히 나누었습니다.

2. Lua 통합 및 실시간 워크플로우

Lua 통합은 FLuaScriptSystem과 ULuaScriptComponent를 중심으로 구현했습니다. 엔진은 **LuaJIT**와 **sol2**를 사용하여 Lua 5.1 호환 런타임을 구동합니다.

- **컴포넌트 구조:** ULuaScriptComponent는 Actor에 부착되어 BeginPlay 시 스크립트를 로드하고, 매 Tick마다 Lua의 Tick(owner, deltaTime)을 호출하며 EndPlay에서 정리 작업을 수행합니다.
- **템플릿 제공:** template.lua를 통해 OnOverlap, OnHit, OnInteract 등 공통 이벤트 함수를 제공하여, 새 스크립트 생성 시 이를 복사해 빠르게 작업할 수 있도록 했습니다.
- **생산성 도구:** 파일의 수정 시간을 주기적으로 확인하여 변경 감지 시 ReloadScript를 호출하는 **Hot Reload** 기능을 구현했습니다. 덕분에 에디터에서 스크립트를 수정하고 즉시 게임에 반영할 수 있었습니다.

3. API 바인딩 및 상태 공유

스크립트가 월드와 유기적으로 상호작용할 수 있도록 로직 위임 기준의 API를 구성했습니다.

- **월드 상호작용:** FindActorByName, GetPlayerActor, RaycastCenter 등을 통해 스크립트에서 월드 내 객체를 탐색할 수 있습니다.

- **상태 공유:** SetGameState와 GetGameState를 통해 여러 Lua 스크립트가 데이터를 공유합니다. 예를 들어 Mop.lua에서 대걸레의 오염도를 기록하면 다른 시스템에서 이를 참조해 시각적 변화를 줄 수 있습니다.
 - **시스템 호출:** 사운드 재생, UI 상태 설정, 아이템 등록 등 게임 전반의 시스템을 Lua에서 직접 제어할 수 있도록 바인딩했습니다.
-

4. 코루틴(Coroutine)과 시간 제어

Lua의 시간 기반 흐름 처리를 위해 FLuaCoroutineScheduler를 별도로 구현했습니다.

- **스케줄링:** wait(seconds) 또는 yield(seconds) 호출 시 C++ 스케줄러가 해당 코루틴을 대기시키고, 정해진 시간이 지나면 엔진 Tick에서 자동으로 재개합니다.
 - **활용 사례:** BeginPlay에서 순차적인 로직을 실행하거나, 엔딩 연출 시 대화가 끝날 때까지 특정 간격으로 대기하는 흐름을 자연스럽게 작성할 수 있었습니다.
-

5. Delegate를 활용한 이벤트 위임

"확실한 위임"이라는 주제에 맞춰 가변 인자 템플릿 기반의 TDelegate를 구현했습니다.

- **안정성:** Broadcast 호출 시 IsUObjectAlive로 객체의 생존 여부를 확인하여 파괴된 객체의 콜백을 안전하게 정리합니다.
 - **충돌 연결:** UPrimitiveComponent의 충돌 이벤트(Overlap, Hit)를 ULuaScriptComponent가 구독하여 Lua 함수로 넘겨줍니다. 이를 통해 C++의 물리 판정과 Lua의 게임 규칙이 **느슨하게 연결된(Loosely Coupled)** 구조를 완성했습니다.
-

6. 물리 기반 상호작용 (ShapeComponent)

UShapeComponent를 통해 실제 게임 상호작용의 물리적 기반을 마련했습니다.

- **충돌체 구조:** Box, Sphere, Capsule 등 구체적인 형태를 구현하고, 메시 외형과 별개로 실제 판정을 담당하는 충돌체를 구성했습니다.
- **컴포넌트 결합:** RigidBodyComponent와 결합하여 플레이어가 물건을 집거나 떨어뜨리는 물리적 피드백을 구현했습니다.

7. 게임 로직 및 시스템 구현

Game 폴더 내에서는 청소, 분류, 점수, 엔딩 시스템이 유기적으로 돌아갑니다.

- **GGameContext:** 현재 진행도, 들고 있는 오브젝트, 아이템 상태 등을 저장하는 중앙 컨텍스트입니다.
- **아이템 및 도구 시스템:** JSON 데이터를 기반으로 아이템 정보를 로드하며, 대걸레나 스펀지 같은 도구의 청소 범위 및 성능을 관리합니다.
- **엔딩 및 스코어:** 보관된 아이템의 성격에 따라 엔딩 타입이 결정되며, 청소 점수와 미션 보너스 등을 합산하여 기록을 남깁니다.

8. 요약 및 결론

정리하자면 이번 프로젝트는 "엔진은 실행과 연결을 맡고, 게임 규칙은 Lua와 Delegate로 위임한다"는 방향성을 성공적으로 수행했습니다.

LuaJIT 통합과 Hot Reload로 개발 효율을 극대화했고, 가변 인자 템플릿 기반의 Delegate와 코루틴을 통해 복잡한 게임 로직을 유연하게 처리했습니다. 이러한 구조 덕분에 엔진은 안전하게 기반 기능을 수행하고, 사물별 개성 있는 반응은 가벼운 스크립트로 분리하여 빠른 반복 테스트와 구현이 가능했습니다.